

Brief — Akshada (UI + Testing)

What we're building, in one line

A practice interview app that listens to you speak, notices what your answer was missing, and asks a follow-up about exactly that.

Your job: make it visible on screen, and make sure it doesn't fall over in front of judges.

Why the UI matters more than usual here

The judges can't see our code. They see a screen for about three minutes.

Half the score is "is the idea good" and "is it clear." A judge looking at our screen needs to understand, without us explaining, that **the AI is actually tracking the conversation** — not just recording audio and spitting out a number.

That's one specific component doing most of the work. It's first on the list.

Build 1 — The state card (most important thing you'll build)

After every answer, the AI fills in a little card showing what the answer contained and what it was missing. It updates live, on screen, while the interview runs.

Two versions depending on question type:

Behavioral questions show STAR slots:

Situation	✓	
Task	✓	
Action	×	← asking about this
Result	~	(vague)

Technical questions show different slots:

Definition	✓	
Mechanism	✓	
Tradeoff	×	← asking about this
Experience	~	

Requirements:

- Three states per slot: filled ✓ / vague ~ / missing ×
- The slot currently being probed is highlighted
- It **animates when it updates** — the judge needs to see it change, not just find it already changed

- Visible during the whole interview, not on a results page afterwards

This is the single most convincing thing on screen. If it's clear, judges believe there's real engineering behind it. If it's buried, we lose that.

Build 2 — Delivery strip

A thin bar showing how they're speaking, not what they're saying:

- "um"s per minute
- speaking pace (words per minute)
- pause pattern

Important: these are **descriptions, not scores**. Show "12 fillers/min" — never "delivery: 4/10."

Reason: judging someone's delivery out of 10 punishes nervous people and people speaking English as a second language, which is a lot of our users. We describe, we don't grade. Someone will ask about this at the pitch, so the UI needs to back up the answer.

Also needs a visible off switch. Being able to turn delivery feedback off is part of the pitch.

Build 3 — "Local · Offline" badge

Small badge in the corner, always visible. Says the model is running on this machine and nothing is going out to the internet.

Sounds trivial. It's the entire premise of the project. During the demo we turn the wifi off on camera — the badge is what makes that moment land visually.

Build 4 — End-of-interview screen

Pick one (your call, tell the group which):

Option A — replay the moments. "Here's where you hesitated on question 3." Click, hear yourself back. Nobody has ever heard themselves in an interview. It's memorable, but it's more work.

Option B — the slot grid. All questions side by side, showing which slot you left empty every time. Most people have one consistent blind spot — usually Result. Simpler to build, genuinely useful.

Option B if time is tight. It's honest and it ships.

Build 5 — Demo mode

A flag that cuts the interview to 2 questions: the warm-up plus one real question where a follow-up fires.

A real interview is 30–45 minutes. Our judging slot is a few minutes. **We must not discover**

this at pitch time.

Build 6 — Delete the login

The competition rules say the demo must work with no login and no paywall. The current auth is a placeholder anyway (one hardcoded user).

Rip it out completely. Straight to the interview. This isn't optional — a login screen is a disqualification risk.

Testing — the part that saves the demo

The bug most likely to exist right now

Skip a round partway through and see if the final report crashes.

This was already flagged in the codebase as the most likely bug in the whole build — the report aggregates scores across rounds, and if a round is missing it may fall over. Test it first. If it crashes and a judge does it, we're finished.

The bug that ruins the scoring

Submit the exact same answer twice. Do the scores match?

If the same answer gets 6 then 8, our grading is random and the whole product is a lie. If they swing, tell the group — it means the reference answers or the settings need fixing, not the UI.

The four things that will happen live

Each needs to not crash:

1. **"I don't know."** → app should accept it, move on, *not* ask a follow-up
2. **Long silence** → after ~10 seconds, offer once: "want me to rephrase?"
3. **"Can you repeat the question?"** → says the *same* question again, doesn't invent a new one
4. **Talking over the AI** → AI stops immediately and listens

A judge will do at least one of these. Probably number 1.

Noise test

Play a YouTube video of a busy room in the background and run an interview. The hackathon hall will be loud.

If it breaks badly, tell the group early — it means we need headset mics, not laptop mics, and we need to know that before the day, not during it.

Don't touch

The innerHTML / XSS security work. There are ~54 spots flagged in the codebase. It's real technical debt, but it's a week of work and no judge will look. Leave it. It's in the writeup as a

known limitation.

Order of work

1. Delete the login (5 minutes, unblocks everything)
 2. State card (the big one)
 3. Demo mode flag
 4. Test: skip-round crash + same-answer stability
 5. Delivery strip
 6. Offline badge
 7. End screen
 8. Test: the four live failure cases
 9. Noise test
-

What "done" looks like

- ☐ No login anywhere
- ☐ State card visible and animating during the interview
- ☐ Both slot types render (STAR + technical)
- ☐ Delivery strip showing numbers, not grades, with an off switch
- ☐ Offline badge visible
- ☐ End screen built
- ☐ Demo mode = 2 questions
- ☐ Skip-round tested, doesn't crash
- ☐ Same answer twice → same score
- ☐ All four failure cases handled
- ☐ Tested with background noise

Report back on two things specifically: whether skip-round crashes, and whether scores are stable. Those two determine whether other people need to change their work.